

COL100: Introduction to Computer Science

Lab 4: Conditionals

January 27, 2025

1 Conditional Statements

A conditional statement allows you to control the flow of execution in a program. It is used to execute different blocks of code based on certain conditions. These conditions are usual logical conditions from mathematics. Conditions can be either **true** or **false**. For example

- Less than or equal to: $a \leq b$
- Greater than: $a > b$
- Equal to: $a == b$
- Not equal to: $a \neq b$

Conditions can be combined using **boolean operators**.

- **and** operator represents logical AND
- **or** operator represents logical OR
- **not** operator represents logical NOT

For example, `condition1` and `condition2` can be combined as '`condition1 and condition2`' or '`condition1 or condition2`'. NOT of `condition1` can be written as `not condition1`.

1.1 if, elif, else Statements

1.1.1 if Statement

The most basic conditional statement '`if`' is used to execute a block of code only if the specified `condition` is true. Look at the following snippet for syntax.

```
1 if (condition) :  
2     # block of code to be executed if the condition is true
```

1.1.2 else Statement

The 'else' statement is used to specify an alternative block of code to be executed if the condition is false. Look at the following snippet for syntax.

```
1 if (condition) :  
2     # block of code to be executed if the condition is true  
3 else :  
4     # block of code to be executed if the condition is false
```

1.1.3 elif Statement

In addition to the 'if' and 'else' statements, you can use the 'elif' statement to specify multiple conditions to be checked in sequence. This is useful when you have more than two possible outcomes. Look at the following snippet for syntax.

```
1 if (condition1) :  
2     # block of code to be executed if condition1 is true  
3 elif (condition2) :  
4     # block of code to be executed if the condition1 is false and  
5     # condition2 is true  
6 else :  
7     # block of code to be executed if the condition1 is false and  
8     # condition2 is false
```

You can use elif to create a ladder of if-elif. This is useful when you have multiple control flow.

```
1 if (condition1) :  
2     # block of code to be executed if condition1 is true  
3 elif (condition2) :  
4     # block of code to be executed if condition2 is true and previous  
5     # conditions are false  
6 elif (condition3) :  
7     # block of code to be executed if condition3 is true and previous  
8     # conditions are false  
9 ...  
10 ...  
11 ...  
12 else :  
13     # block of code to be executed if all conditions are false
```

1.1.4 Nested if-else

Nested if statements mean an if statement inside another if statement. Take a look at the following snippet to understand

```
1 if (condition1) :  
2     # Executes when condition1 is true  
3     if (condition2) :
```

```
4     # Executes when condition2 is true
5     else :
6         # Executes when condition2 is false
```

1.1.5 Example Program: Largest of the Three

Here is an example program that takes three integers as input and outputs the largest integer.

```
1 if __name__ == "__main__":
2     # Input three numbers
3     print("Enter three numbers")
4
5     num1 = int(input())
6     num2 = int(input())
7     num3 = int(input())
8
9     # Compare and find the largest
10    if (num1 >= num2 && num1 >= num3) :
11        print("The largest number is:", num1)
12    elif (num2 >= num1 && num2 >= num3) :
13        print("The largest number is:", num2)
14    else :
15        print("The largest number is:", num3)
```

2 Submission Problem

2.1 Instructions

- Complete the code of the following problem in **main.py** program file.
- Submit the program file in the assignment titled “Lab 4” on Gradescope.
- The Autograder on Gradescope will evaluate the submitted program on some test cases.

2.2 Leap Year Checker

Write a program that takes a (integer) year as input and determines whether it is a leap year. Your program must output **YES** if the input year is a leap year, else it must output **NO**. Your program should also be able to handle cases where the input is not a valid year value or if it is not an integer at all. In such cases, it can output *INVALID YEAR*.

Sample Input 1

```
1 2004
```

Sample Output 1

```
1 YES
```

Sample Input 2

```
1 -5
```

Sample Output 2

```
1 INVALID YEAR
```

3 Practice Problems

3.1 Grading System

Write a program that takes a student's numerical score (integer) as input and outputs their corresponding letter grade based on the following scale:

- A: 90-100
- B: 80-89
- C: 70-79
- D: 60-69
- F: Below 60

Handle invalid input cases, such as scores below 0 or above 100 by printing *INVALID SCORE*.

Sample Input 1

```
1 78
```

Sample Output 1

```
1 Letter Grade: C
```

3.2 Electricity Bill Calculator

Write a program to calculate and print the electricity bill of a given customer. The program should take the (integer) units consumed by the user from the keyboard as input and output the total amount to be paid by the customer. The charges are as follows:

Unit Range	Charge per Unit
upto 199	Rs. 1.20
200 and above but less than 400	Rs. 1.50
400 and above but less than 600	Rs. 1.80
600 and above	Rs. 2.00

If the bill exceeds Rs. 400, then a surcharge of 15% will be added. Additionally, the minimum bill amount should be Rs. 100.

Sample Input 1

```
1 0
```

Sample Output 1

```
1 100
```

Sample Input 2

```
1 100
```

Sample Output 2

```
1 120
```

Sample Input 3

```
1 600
```

Sample Output 3

```
1 1380
```

3.3 Quadratic Equation Solver

Write program that takes three (floating point) coefficients a , b , and c of a quadratic equation ($ax^2 + bx + c = 0$) as input and outputs for its roots unto to two decimal places. Your program should handle the following cases:

- If the equation has two distinct real roots, output both roots.
- If the equation has one real root (repeated root), output the root.
- If the equation has no real roots, output the message *IMAGINARY ROOTS*.

Sample Input 1

```
1 2.00 5.00 2.00
```

Sample Output 1

```
1 Root 1: -0.50
2 Root 2: -2.00
```

Sample Input 2

```
1 1.00 -6.00 9.00
```

Sample Output 2

```
1 Root: 3.00
```

Sample Input 3

```
1 2.00 4.00 8.00
```

Sample Output 3

```
1 IMAGINARY ROOTS
```

3.4 Rock Paper Scissors Game

Get ready because you are now going to build your first computer game! We will be building a rock paper scissors game where people can play against the computer.

Users can enter their choice according to the following convention.

- 1 for Rock
- 2 for Paper
- 3 for Scissors

You are given two integer variables *player1* and *player2*, representing the choice of the first player and the second player, respectively. The variable *ans* represents who wins and should be populated according to the following scheme.

- 1 if the first player wins
- 0 if it is a tie
- -1 if the second player wins

Task: You need to write and test a function *decide_answer* which returns the value of *ans* according to the values of *player1* and *player2*.